LABORATORY OBSERVATION

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 4
Student Name: M.Purna Chandra
Roll No.: A24126552095
Date: 16-08-2026

---

1. TITLE

Develop a To-Do List Application Using JavaScript DOM Manipulation

---

2. AIM

To design and develop a To-Do List web application that uses JavaScript DOM manipulation to dynamically create, complete, and delete tasks, and to conditionally display an empty-state message.

---

3. OBJECTIVE

The objectives of this experiment are:

- To understand and use document.createElement() to dynamically build DOM elements.
- To append and remove child nodes using appendChild() and remove().
- To handle user interaction using addEventListener() for click and keypress events.
- To capture and validate text input, trimming whitespace and preventing empty entries.
- To toggle CSS classes dynamically using classList.toggle() for visual state changes.
- To conditionally show or hide an element using style.display based on list state.
- To style task items, buttons, and states using CSS3 Flexbox and transitions.

---

4. THEORY

DOM Element Creation — document.createElement() builds a new, detached element in memory; textContent sets its visible text, and className assigns a CSS class for styling.

DOM Tree Manipulation — appendChild() inserts an element as the last child of a parent node, allowing new elements to be composed and attached to the page at runtime; remove() detaches an element from the DOM entirely.

Event Listeners — addEventListener() attaches a function that runs when a specified event, such as click or keypress, occurs on an element, enabling interactive behaviour without inline handlers.

classList.toggle() — adds a CSS class to an element if it is not already present, and removes it if it is, making it well suited to on/off visual states such as a completed task.

CSS Flexbox — display: flex arranges the input field and Add Task button in a single row, and arranges each task's text and action buttons within the list item.

---

5. ALGORITHM / PROCEDURE

- Create three files: index.html, style.css, and script.js.
- In index.html, add a text input, an Add Task button, an empty <ul id="taskList">, and an empty-state <p id="emptyMessage">.
- Link style.css in <head> and script.js before the closing </body> tag.
- In script.js, select the input, button, task list, and empty-message elements with getElementById().
- Write updateEmptyMessage() to show the empty message when taskList has no children, and hide it otherwise.
- Write addTask() to read and trim the input value, and return early if it is empty.
- Inside addTask(), create a <li>, a for the task text, and a to hold the Complete and Delete buttons using createElement().
- Set the className and textContent of each created element, then assemble them with appendChild().
- Add a click listener on the Complete button to toggle the completed class on the <li> using classList.toggle().
- Add a click listener on the Delete button to remove the <li> from the DOM and call updateEmptyMessage().
- Append the finished <li> to the task list, clear the input field, and call updateEmptyMessage().
- Attach a click listener on the Add Task button and a keypress listener on the input to call addTask() on Enter.
- Call updateEmptyMessage() once on page load to set the initial state.
- In style.css, style the container, input group, task items, and completed/complete/delete states with Flexbox and colour.
- Open index.html in a browser, add, complete, and delete tasks, and verify the empty-state message toggles correctly.
- Record expected vs. actual output for each test case.

---

6. PROGRAM

index.html

<!DOCTYPE html>

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My To-Do List</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <main class="todo-container">
    <h1>My To-Do List</h1>

    <div class="input-group">
      <input type="text" id="taskInput" placeholder="Enter a task..." />
      <button id="addTaskBtn">Add Task</button>
    </div>

    <!-- Container for dynamic task elements -->
    <ul id="taskList"></ul>

    <!-- Empty state message -->
    <p id="emptyMessage" class="empty-message">No tasks available.</p>
  </main>

  <script src="script.js"></script>
</body>
</html>
```

style.css

```
/* Minimal Reset & Centered Layout */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
```

```
}

body {
  background-color: #f8f9fa;
  color: #212529;
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

.todo-container {
  background: #ffffff;
  padding: 2rem;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.05);
  width: 100%;
  max-width: 480px;
}

h1 {
  font-size: 1.5rem;
  font-weight: 600;
  margin-bottom: 1.5rem;
}

/* Input Area */
.input-group {
  display: flex;
  gap: 0.5rem;
  margin-bottom: 1.5rem;
}

input[type="text"] {
  flex: 1;
```

```
  padding: 0.6rem 0.8rem;
  border: 1px solid #dee2e6;
  border-radius: 4px;
  font-size: 0.95rem;
  outline: none;
}

input[type="text"]:focus {
  border-color: #4da6ff;
}

button {
  padding: 0.6rem 1rem;
  border: none;
  border-radius: 4px;
  font-size: 0.9rem;
  cursor: pointer;
  transition: opacity 0.2s;
}

button:hover {
  opacity: 0.85;
}

#addTaskBtn {
  background-color: #0066cc;
  color: white;
}

/* Task List Items */
ul {
  list-style: none;
}

.task-item {
```

```
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.75rem 0;
  border-bottom: 1px solid #e9ecef;
  gap: 0.5rem;
}

.task-text {
  flex: 1;
  font-size: 0.95rem;
  word-break: break-word;
}

/* Visual state for completed tasks */
.completed .task-text {
  text-decoration: line-through;
  color: #868e96;
}

/* Action Buttons */
.btn-group {
  display: flex;
  gap: 0.4rem;
}

.complete-btn {
  background-color: #e6f4ea;
  color: #137333;
}

.delete-btn {
  background-color: #fce8e6;
  color: #c5221f;
}
```

```
/* Empty State Message */
.empty-message {
  color: #868e96;
  font-size: 0.9rem;
  text-align: center;
  margin-top: 1rem;
}
```

script.js

```
// 1. Select DOM Elements
const taskInput = document.getElementById('taskInput');
const addTaskBtn = document.getElementById('addTaskBtn');
const taskList = document.getElementById('taskList');
const emptyMessage = document.getElementById('emptyMessage');

// Helper function to update the visibility of the "No tasks available" message
function updateEmptyMessage() {
  if (taskList.children.length === 0) {
    emptyMessage.style.display = 'block';
  } else {
    emptyMessage.style.display = 'none';
  }
}

// Function to handle adding a new task
function addTask() {
  const taskText = taskInput.value.trim();

  // Prevent adding empty tasks
  if (taskText === '') return;

  // 2. Dynamic Element Creation
  const li = document.createElement('li');
  li.className = 'task-item';
```

```
const textSpan = document.createElement('span');
textSpan.className = 'task-text';
textSpan.textContent = taskText;

const btnGroup = document.createElement('div');
btnGroup.className = 'btn-group';

const completeBtn = document.createElement('button');
completeBtn.className = 'complete-btn';
completeBtn.textContent = 'Complete';

const deleteBtn = document.createElement('button');
deleteBtn.className = 'delete-btn';
deleteBtn.textContent = 'Delete';

// Assembly of task item components
btnGroup.appendChild(completeBtn);
btnGroup.appendChild(deleteBtn);
li.appendChild(textSpan);
li.appendChild(btnGroup);

// 3. Dynamic Modification of DOM & Event Listeners

// Toggle complete state visually
completeBtn.addEventListener('click', function () {
  li.classList.toggle('completed');
});

// Remove task item from DOM
deleteBtn.addEventListener('click', function () {
  li.remove();
  updateEmptyMessage(); // Check if list is empty after removal
});

// Append new task to the list
```

```
  taskList.appendChild(li);

  // Clear input field and refresh empty state
  taskInput.value = '';
  updateEmptyMessage();
}

// 4. Event Listeners for User Interaction
addTaskBtn.addEventListener('click', addTask);

// Support pressing "Enter" key to add a task
taskInput.addEventListener('keypress', function (e) {
  if (e.key === 'Enter') {
    addTask();
  }
});

// Initial check on page load
updateEmptyMessage();
```

---

7. CODE EXPLANATION

Code	Explanation
document.createElement()	Creates a new, empty DOM element of the given tag name.
textContent	Sets the visible text inside an element without parsing HTML.
appendChild()	Inserts a child node at the end of a parent element's contents.
addEventListener('click', ...)	Registers a function to run when the element is clicked.
addEventListener('keypress', ...)	Registers a function to run when a key is pressed inside the input.
classList.toggle('completed')	Adds the completed class if it is absent, removes it if present.
remove()	Deletes the element from the DOM.
style.display	Directly sets the inline display style to show or hide an element.
trim()	Removes leading and trailing whitespace from the input text.
display: flex (.input-group)	Places the text input and Add Task button in a single row.
button:hover	Reduces the button's opacity slightly on mouse-over.

Code	Explanation
.completed .task-text	Applies a line-through style and muted colour to completed task text.

---

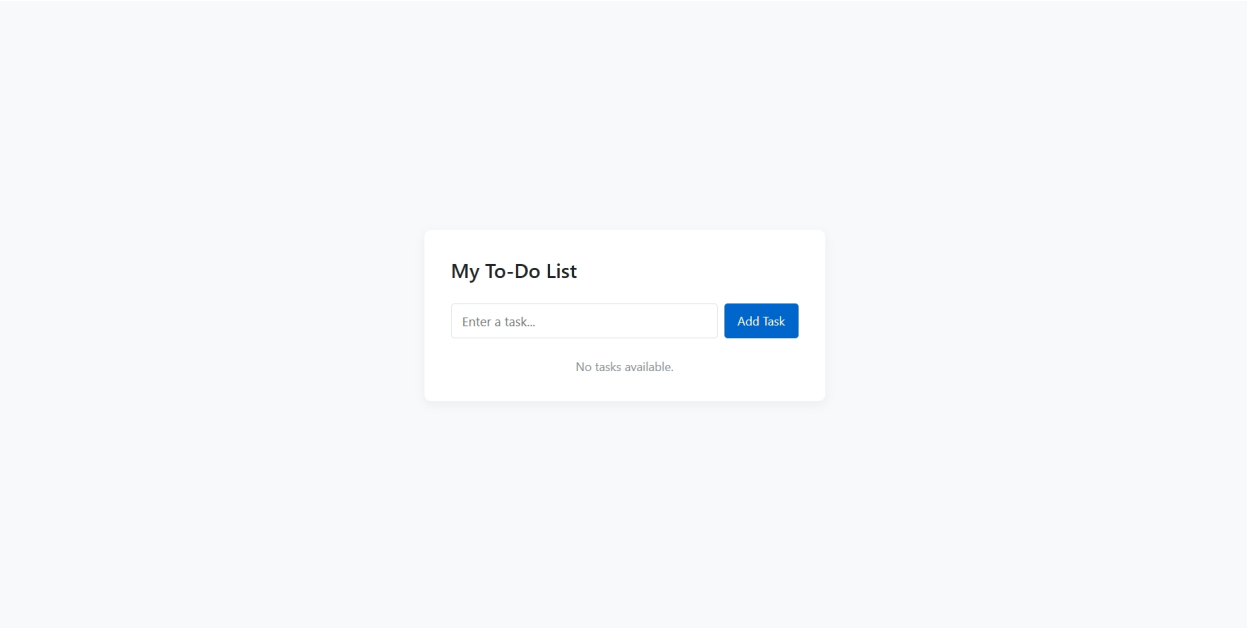
8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Load the page in a browser	Task list is empty; "No tasks available." is shown	Empty message displayed correctly on load	Pass
TC-02	Click Add Task with an empty input	No task is added; empty message remains visible	No task added; input stayed empty	Pass
TC-03	Type a task and click Add Task	Task appears in the list; empty message hides	Task added; empty message hidden	Pass
TC-04	Type a task and press Enter	Task is added the same way as clicking the button	Task added on Enter key press	Pass
TC-05	Click Complete on a task	Task text shows a strikethrough style	Strikethrough applied to task text	Pass
TC-06	Click Complete again on the same task	Strikethrough is removed (state toggles off)	Strikethrough removed correctly	Pass
TC-07	Click Delete on a task	Task is removed from the list	Task removed from the DOM	Pass
TC-08	Delete all tasks from the list	"No tasks available." message reappears	Empty message reappeared after last delete	Pass
TC-09	Add several tasks in sequence	All tasks appear in the order they were added	Tasks listed in correct order	Pass
TC-10	Hover over Add Task, Complete, or Delete buttons	Button opacity reduces slightly on hover	Opacity transition applied on hover	Pass

---

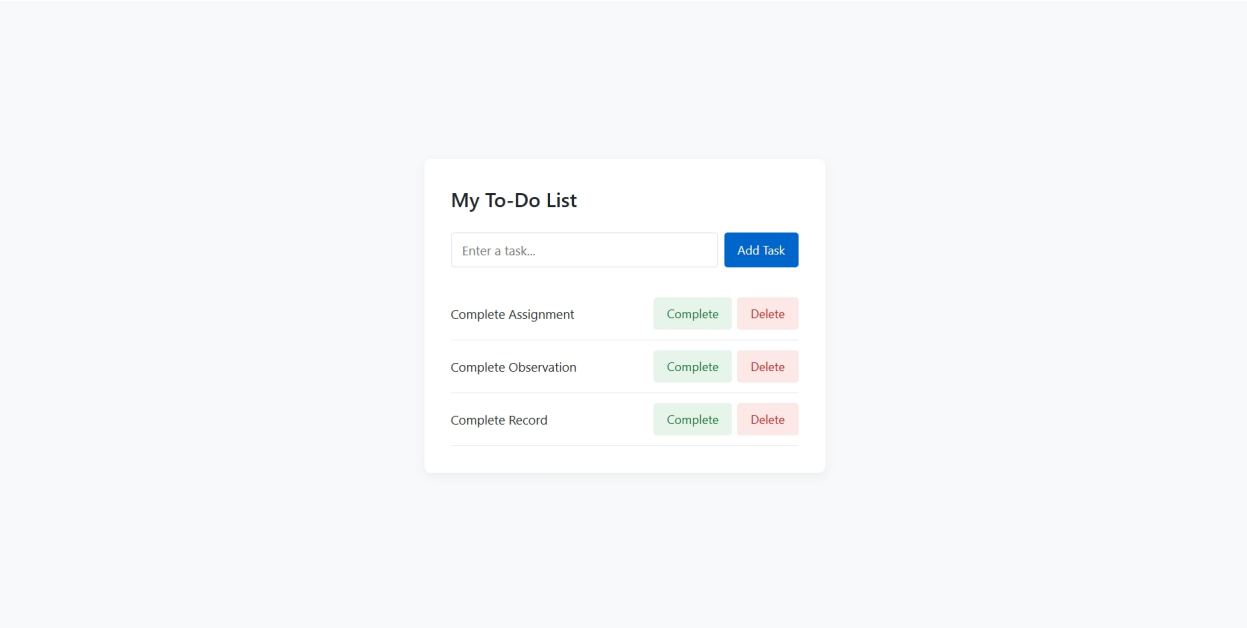
9. OUTPUT

Output 1: Initial Page Load



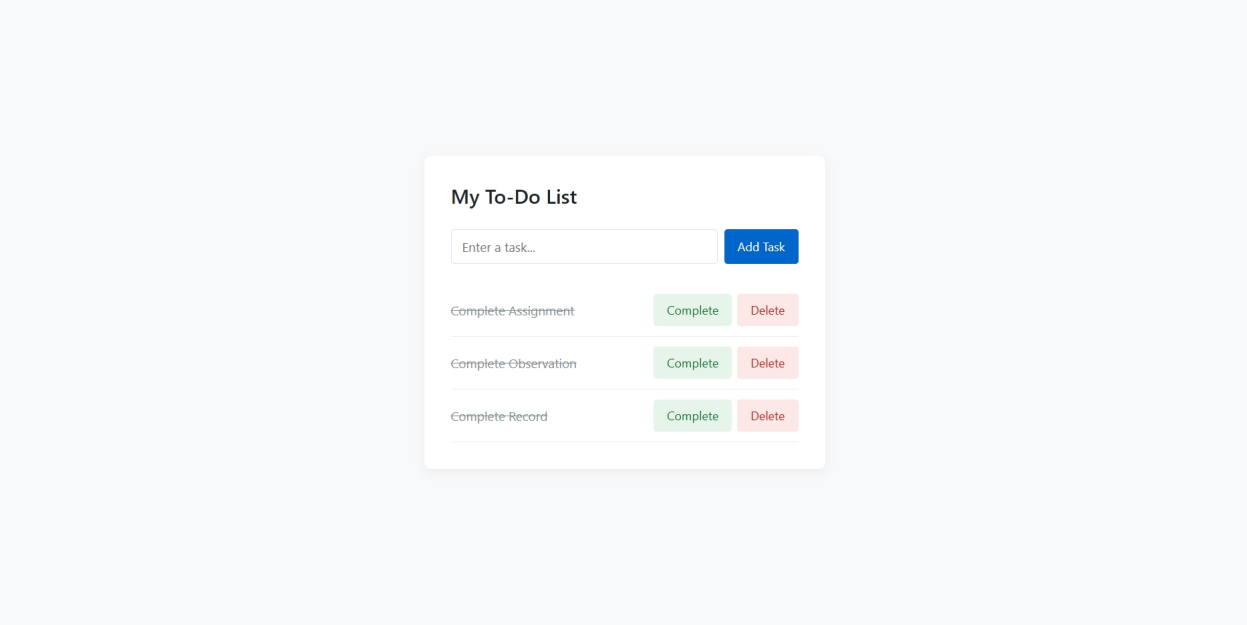
The page renders the input field, Add Task button, and an empty task list, with the "No tasks available." message shown below.

Output 2: Task List With Multiple TasksSeveral tasks have been added and appear in the list, each with Complete and Delete buttons; the empty-state message is hidden.



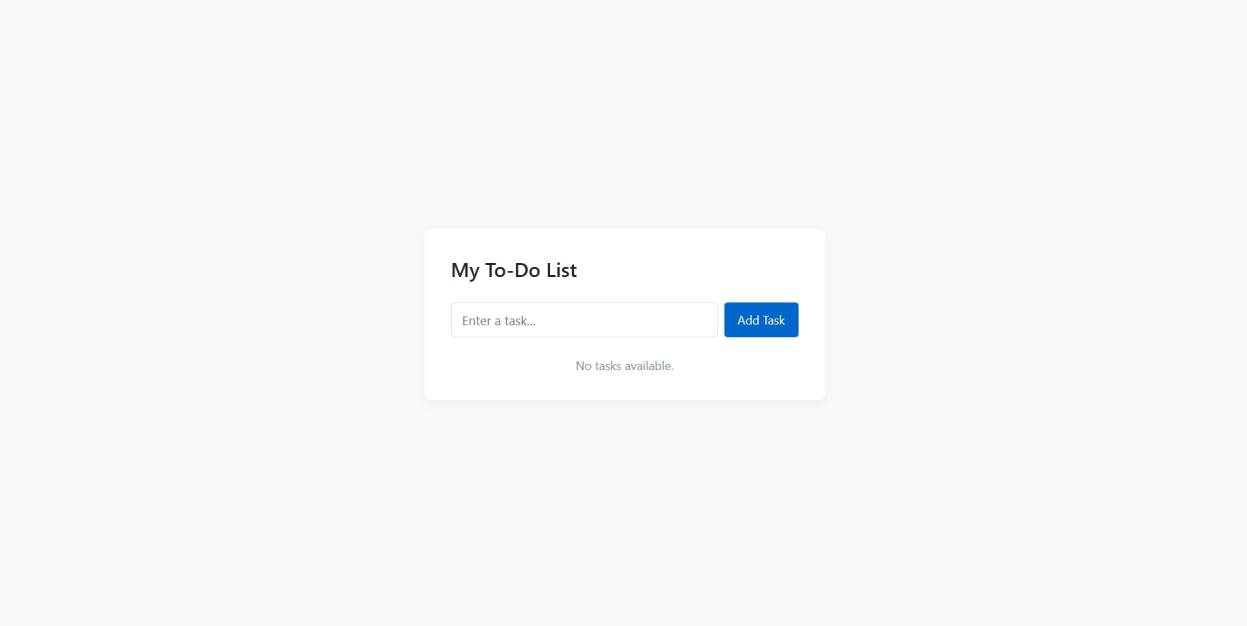
Output 3: Completed Task

A task marked as complete shows a strikethrough style on its text, indicating its toggled state.



Output 4: Empty State After Deleting All Tasks

After deleting every task from the list, the "No tasks available." message reappears below the empty list.



---

10. RESULT

Thus, a To-Do List web application was successfully designed and developed using JavaScript DOM manipulation. The application demonstrates dynamic element creation, event-driven task completion and deletion, and conditional display of an empty-state message. All ten test cases passed successfully, confirming correct task creation, state toggling, deletion, and empty-state behaviour.